

Saints Club Limited – Developer

Brief & Work Plan

Sprint: 2 working days

Demo build: <https://saints-club.vercel.app/>

Prepared by: Project Manager (non-technical)

Audience: Lead developer

0. Read this first (PM note)

The Proposed Scope of Work has six workstreams. **Four of them cannot be finished in two days** – not because of developer speed, but because they depend on third parties who take days or weeks to approve access:

Workstream	2-day feasible?	Blocker
1. Public website (6 sections)	Yes	Final content + logo needed
2. Member database + portal + admin field visibility	Yes (core)	Clean data file needed
3. Social media live feed	No	Meta App Review + Business Verification

Workstream	2-day feasible?	Blocker
		takes 1–4 weeks. X API is paid.
4. E-commerce + bKash	No	bKash merchant onboarding + sandbox credentials take 1–3 weeks
5. Restaurant / vendor API	No	No restaurant POS system identified yet
6. Admin, security, training	Partial	Basic roles + SSL in sprint; training material later

So this sprint delivers items 1 and 2. Sections 3–5 are scoped in Part D as Phase 2/3, with the paperwork you must start *today* so the developer is not blocked later.

Tell the client this plainly. Promising a bKash-integrated e-commerce club portal in 48 hours will damage credibility when it slips.

PART A — Sprint scope

A1. In scope (must ship by end of Day 2)

1. Six-section public website, fully responsive, matching the approved sitemap and copy.

2. Digital Notice Board (admin can post announcements; they appear on the site).
3. Member database imported and searchable.
4. Member login + "My Profile" page.
5. **Admin-controlled field visibility** — the feature described in Part B. This is the highest-value item in the sprint.
6. Role-based admin panel (Super Admin / Secretariat / Committee Viewer).
7. CSV/Excel export of member data for the Board.

A2. Explicitly out of scope this sprint

- Payment of any kind (bKash, card, bank)
- Merchandise store
- Live social media feed
- Restaurant ordering
- Bangla translation (build the site *translation-ready*, don't translate yet)
- Online membership application with document upload (Phase 2 — a simple enquiry form is enough now)

A3. Definition of done

The client can log in as admin, tick which member fields are visible to whom, log out, log in as an ordinary member, and see exactly the fields the admin allowed — nothing more. Everything else in the sprint is supporting work for that demo.

PART B – The field visibility feature (spec)

This is the requirement written in the scope as *"Admin can show additional information of Members based on their individual choice."* It is ambiguous. Build it as **two independent layers**, because both readings are legitimate and the club will want both.

Layer 1 – Club policy (set by Admin, applies to everyone)

Admin sees a grid: **rows = fields, columns = audiences**.

Field	Public (logged out)	Members (logged in)	Committee	Admin only
First / Last name	☒	☒	☒	☒
Membership Type	☐	☒	☒	☒
Membership Status	☐	☒	☒	☒
Batch	☐	☒	☒	☒
Institute	☐	☒	☒	☒
Job Title	☐	☒	☒	☒

Field	Public (logged out)	Members (logged in)	Committee	Admin only
Organisation	☐	☒	☒	☒
Mobile	☐	☐	☒	☒
Email	☐	☐	☒	☒
Marital Status	☐	☐	☐	☒
Marriage Anniversary	☐	☐	☐	☒
Date of Birth	☐	☐	☐	☒
Blood Group	☐	☐	☒	☒
Gender	☐	☐	☐	☒
Home Address	☐	☐	☐	☒
Nationality	☐	☐	☐	☒
NID number	☐	☐	☐	☐ (never displayed – see B4)

Ship with the defaults shown above (closed by default). It is far easier to explain to the Board why a field is hidden than to

explain why 300 members' home addresses were public for a week.

Layer 2 — Member consent (set by each member on their own profile)

Each member gets toggles on "My Profile": *Show my mobile to other members* / *Show my email* / *Show my organisation* / *Show my blood group*.

The resolution rule (write this as one function and test it)

```
visible(field, viewer) =  
  club_policy_allows(field, viewer.role)  
  AND member_consent_allows(field, viewer.role)  
  AND NOT field.is_never_public
```

Both layers must say yes. Admin policy can *restrict* but never *override* a member who has opted out. Admin role itself always sees everything (it is the club's own record).

B4. Fields that must never be renderable, at any setting

- NID / National ID number
- Password hashes, session tokens
- Internal admin notes

These are stored (NID for verification) but must be excluded at the query layer, not just hidden in the UI. Do not send them to the browser at all.

B5. Implementation notes for the developer

- Store policy in a `field_visibility_policy` table:
(`field_key`, `audience`, `is_visible`, `updated_by`, `updated_at`) — not in a config file. The client must change it without a deploy.
 - Store consent in `member_field_consent`: (`member_id`, `field_key`, `audience`, `is_visible`).
 - **Filter server-side.** Build a `serializeMember(member, viewer)` function that strips hidden fields before the response leaves the server. Never send the full record and hide with CSS or client-side JS — that is a data leak with extra steps.
 - Every policy change writes an audit row: who changed what, when, old value, new value. The Board will ask.
 - Search must respect visibility too. If mobile numbers are hidden from members, a member must not be able to find someone *by* searching a mobile number.
 - Exports respect the exporting user's role. A Committee Viewer export must not contain admin-only columns.
-

PART C — Data, schema, and the member file

C1. Problems found in

`member_data__07_Agust_26__Hira` (flag to client)

I reviewed the file. **Do not import it as-is.** Issues:

1. **Two different file formats are concatenated.** The main block has 20 columns (Member ID ... Membership Status). The final block has a different schema (Name, Batch, Institute, Male/Female, DOB, **NID**, Membership Type, Mobile) and no Member IDs. These must be reconciled before import.
2. **Excel corruption in the second block.** Some NID values show as 4.47826E+11 (scientific notation) and some dates show as ##### (column too narrow). The original values are lost in this export – request the source Excel file, not a PDF/print.
3. **Placeholder strings used as data:** Need-To-Add , No Email Found , No Contact Found , No Address Found , N/A , UNKNOWN , XX . Import these as NULL, not as text, or the directory will display "Need-To-Add" as someone's job title.
4. **Fake dates of birth.** 1901-01-01 is used as an "unknown DOB" placeholder across many records. Also present: a DOB of 2017-03-28 and one of 2024-09-22 on adult members – clearly data-entry errors. Import 1901-01-01 as NULL; flag the future/child dates for manual correction.
5. **Whitespace loss.** A large run of Life Member rows has all spaces stripped (Md.SayeedHossainLMmale1979-12-05married...). These rows will not parse with a naive splitter. Get a clean CSV export.
6. **Duplicate contact numbers** across different members (e.g. the same mobile appearing on two separate Life Member records). Deduplicate and confirm which is correct.

7. **Corporate entities stored as people** (club/company names in the person-name field, gender `not_applicable`). Model these as an `organisation` member type with named nominees, per the Corporate Membership rule (up to 3 C-Suite nominees).

C2. Institute mismatch — a business question, not a bug

The Member ID suffix encodes the school: `-SG` St. Gregory's, `-SJ` St. Joseph, `-HC` Holy Cross, `-SF` St. Francis Xavier's, `-SP` St. Placid's, `-SS` St. Scholastica's, `-GI` Greenherald. These are exactly the seven eligible institutions.

But a large group of records carry the suffix `-CU` and list institutes that are *not* on the eligible list — Notre Dame College, Bottomely Home Girls' High School, Udayan School, St. Mary's, Sacred Heart, Fatima High School.

PM action: ask the Board directly — are these legacy members from an absorbed club (a "Club Notre Damians BD Ltd." entity appears in the data), or are they mis-tagged? The answer changes the eligibility validation rule and the quota maths. Do not let the developer guess.

C3. Membership type codes to confirm

The data uses codes; the scope document uses names. Get the Board to sign off this mapping before import:

Code	Probable meaning	Confirm?
FM	Founder Member	☐
DM	Donor Member	☐
LM	Life Member	☐
PM	Permanent Member	☐
PT	Patron Member	☐
HM	Honorary Member	☐
NR	Non-Resident Member	☐
AS	Associate Member	☐
SM	Senior Member	☐
CR	Corporate Member	☐
CNBL / NONM / SCL	Organisational entities	☐

Note: **"Patron" appears in the fee table (BDT 7,00,000) but is missing from the membership category list** in the website copy. The two documents disagree. Get one authoritative version before the Membership page is built.

C4. Quota tracking

Enforce in the schema and show live counts on the admin dashboard:

- Total General Members ceiling: **2,318**

- Donor: max 300 · Life: max 500 · Permanent: max 1,500
- Block new approvals when a category is full; show "X of Y used" to admin.

C5. Recommended stack

The demo is on Vercel, so stay in that lane — no time to change platforms:

- **Next.js (App Router) + TypeScript**
- **PostgreSQL** — Supabase or Neon (both have free tiers, both work with Vercel)
- **Prisma** ORM
- **NextAuth / Auth.js** — email + OTP login for members, password for admin
- **Tailwind + shadcn/ui** — do not hand-build components in a 2-day sprint
- **Resend or SMTP** for transactional email

Constraints: server-side rendering for anything showing member data; no member data in client bundles; environment variables for all secrets; database in a region close to Bangladesh (Singapore) for latency.

C6. Data model sketch

```
members          (id, member_code, first_name,
last_name, member_type_id,
                  institute_id, batch_year,
gender, dob, marital_status,
                  anniversary, blood_group,
nationality, job_title,
```

```

organisation, job_location,
address, status, created_at)
member_contacts (id, member_id,
type[phone|email], value, is_primary)
member_types (id, code, name,
admission_fee, subscription_fee,
subscription_period,
quota_max, has_voting_rights)
institutes (id, code, name, city,
founded_year, is_eligible)
users (id, member_id, email, role,
last_login)
roles (super_admin, secretariat,
committee_viewer, member)
field_visibility_policy (field_key, audience,
is_visible, updated_by, updated_at)
member_field_consent (member_id, field_key,
audience, is_visible)
notices (id, title, body, category,
published_at, published_by)
audit_log (id, actor_id, action, entity,
before, after, at)

```

PART D — Day-by-day plan

Assume one developer, ~8 focused hours per day. Items marked ■ are the ones that will slip first if anything goes wrong — protect them.

Day 1 — Data and admin (the hard half)

Time	Task	Output
0.5h	Repo, env, Postgres provisioned, Prisma connected	Deployed empty app on Vercel
1.5h	Schema from C6, migrations, seed member types + 7 institutes	Migration files
2.0h	■ Import script for the member file, with the cleaning rules in C1	Members in DB + data quality report (see below)
1.5h	Auth: admin login, member login, role middleware	Two working logins
2.0h	■ Field visibility policy grid – admin UI + <code>serializeMember()</code>	Admin can tick boxes and it changes what a member sees
0.5h	Audit logging on policy changes	Audit table populating

Day 1 deliverable to me by end of day: a short data-quality report – total rows imported, rows rejected and why, count per membership type, count per institute, list of duplicate contacts, count of records with missing email/phone/DOB. I need these numbers for the client meeting.

Day 2 – Public site and member experience

Time	Task	Output
2.0h	Public pages: Home, About Us, Governance & Legal, Contact	Four pages live with final copy
1.5h	Membership page: categories, eligibility, fee table, process	Fee table renders correctly (see C3 warning)
1.0h	Club Life & Facilities page	Page live
1.0h	Notice Board: admin post/edit/publish → public list	Working end to end
1.5h	Member directory (search + filter by batch/institute/type) and My Profile with consent toggles	Visibility rules provably enforced
0.5h	Export to CSV/Excel, role-respecting	Download works
0.5h	Responsive check, SSL, meta tags, 404 page, favicon	Passes checklist in Part E

Stretch only if ahead: social media links in the footer (static, not a live feed); a placeholder "Store – coming soon" page.

PART E – Acceptance test (I will run this myself)

Run through it in front of me on Day 2 afternoon:

1. Log out. Open the member directory. → Only names visible, nothing else.
2. Log in as an ordinary member. → See job title and organisation; mobile and address hidden.
3. As admin, tick "Mobile → visible to Members". Save.
4. Refresh the member view. → Mobile now appears.
5. As that member, toggle "Do not show my mobile" on My Profile.
6. Refresh from another member's account. → Mobile hidden again, even though club policy allows it. **This is the critical test.**
7. Open browser DevTools → Network tab on the member directory. → Confirm hidden fields are absent from the API response, not merely hidden visually. **If they appear here, the feature is not done.**
8. Export as Committee Viewer. → No admin-only columns in the file.
9. Search a hidden mobile number. → No result.
10. Post a notice as admin. → Appears on the public Notice Board within one refresh.
11. Open the whole site on a phone. → No horizontal scrolling, tap targets usable.
12. Check the audit log. → Every policy change from steps 3 and 5 is recorded.

PART F — Phase 2 and 3 (and what I must start today)

These are not developer-blocked; they are **paperwork-blocked**. I need to start them now so the developer is not idle in week 2.

Phase 2 — E-commerce + bKash (~2 weeks dev, after credentials arrive)

My action now: apply for a bKash Merchant / PGW account in the club's registered name. Requires trade licence, TIN, bank account details, and an authorised signatory. Ask bKash for **sandbox credentials first** — the developer can build and test against sandbox while live approval is pending. Build: fee invoicing, admission + subscription payments, receipt numbering, reconciliation dashboard, merchandise store.

Phase 3 — Social media live feed (~1 week dev, after Meta approval)

My action now: confirm the club owns the official Facebook Page, Instagram Business account, and X handle. Create a Meta Developer app and start **Business Verification** — this alone can take weeks, and Instagram Graph API needs the account converted to Business and linked to the Page. Interim solution for launch: an embedded page plugin or a scheduled scraper-free widget showing latest posts. Not automatic, but visible.

Phase 4 — Restaurant / vendor ordering

My action now: find out what POS the club restaurant actually uses. If there is no POS, there is no API, and this becomes a build-from-scratch ordering module — a project in its own right, not an "integration". Get this answer before we quote it.

PART G — What I owe the developer before Day 1 starts

Nothing below is optional. If any of it is missing, tell me before you start coding, not on Day 2.

- [] **Clean member data as .xlsx or .csv** (not PDF) — the current file is unusable as-is
- [] Board decision on the **-CU** / non-eligible-institute records (C2)
- [] Signed-off membership type mapping (C3)
- [] Authoritative fee table — including whether "Patron" is a real category
- [] Club logo (SVG or high-res PNG), brand colours, any photography
- [] Final Board of Directors list with names, designations, photos
- [] Real phone number and email for the Contact page (currently **+880-2-XXXXXXX**)
- [] Registered office address for the map

- [] Articles of Association and disciplinary rules as PDFs for the Governance page
 - [] Domain name + DNS access
 - [] Written confirmation from the club that member contact data may be displayed in the directory
-

PART H — Data protection note

The member file contains full names, dates of birth, home addresses, personal mobile numbers, personal emails, blood groups, marital status and — in one section — **National ID numbers**. This is a serious asset and a serious liability.

Minimum standards for this build, non-negotiable:

1. NID numbers are never returned by any API and never rendered. If they are not needed for the launch, **do not import them at all** until the club confirms why they are held.
2. Member data is served over HTTPS only, from server-rendered pages, with role checks on every request.
3. No member data in any client-side bundle, log file, or error report sent to a third party.
4. Database backups encrypted; credentials in environment variables only; nothing in the repo.
5. Get written consent from members before publishing their contact details in a searchable directory — an email announcement with an opt-out window is the practical minimum, and the per-member consent toggles in Part B are how we honour it.

6. Scope the developer's own access: production database credentials should be held by the club, not permanently by the vendor.

Point 5 is a client decision, not a technical one. Raise it at the next Board meeting and get it minuted.

Summary for the client meeting

In two days we will deliver the complete public website, the member database with the admin-controlled privacy system, and the notice board. Payments, the online store, live social feeds and restaurant ordering depend on approvals from bKash, Meta and the restaurant's own systems — we are starting that paperwork now and will build those in Phases 2 to 4. The immediate blocker on our side is a clean copy of the member data file.